



Data Mining in Heuristics

3

Marcelo R. H. Maia, Isabel Rosseti, Simone de Lima Martins,
and Alexandre Plastino

Contents

Introduction	42
Strategies to Use Data Mining in Heuristics	43
DM-MSH and MDM-MSH Strategies	43
MineReduce Strategy	46
Hybrid Heuristics with Data Mining: A Selection of Cases	49
DM-Supported HGS for the Capacitated VRP	49
DM-Supported MS-ILS for the Heterogeneous Fleet VRP	55
DM-Supported MS-ILS for the Multisource Capacitated FLP with Customer Incompatibilities	63
Conclusion	68
Cross-References	68
References	69

Abstract

This chapter explores some heuristics incorporating data mining procedures. The basic idea of using data mining inside a heuristic is to obtain knowledge from previous iterations performed by a heuristic to guide the search in the subsequent iterations. Patterns extracted from good-quality solutions can be used to guide the

M. R. H. Maia

Universidade Federal Fluminense, Niterói, Rio de Janeiro, Brazil

Instituto Brasileiro de Geografia e Estatística, Rio de Janeiro, Brazil

e-mail: [mmaia@ic.uff.br](mailto:mm Maia@ic.uff.br); marcelo.h.maia@ibge.gov.br

A. Plastino · I. Rosseti · S. de Lima Martins

Universidade Federal Fluminense, Niterói, Rio de Janeiro, Brazil

e-mail: rosseti@ic.uff.br; simone@ic.uff.br; plastino@ic.uff.br

search, leading to a more effective exploration of the solution space. This survey shows that heuristics may benefit from data mining by obtaining better solutions in shorter computational times.

Keywords

Data mining · Frequent patterns · Hybrid metaheuristics · Problem decomposition · Problem size reduction

Introduction

Many combinatorial optimization problems of practical relevance are classified as NP-hard, which generally means they cannot be solved efficiently. As a result, one approach to tackling these challenging problems is to accept suboptimal solutions rather than attempting to find the optimal ones. Heuristics serve as computational methods designed to generate and evaluate candidate solutions iteratively. Although the complexity of finding optimal solutions can grow exponentially with the size of NP-hard problem instances, the evaluation of candidate solutions can often be conducted much more efficiently, typically in polynomial time [5].

Metaheuristics have been developed to enhance the performance of heuristics in exploring solution spaces. These high-level procedures provide a framework that guides basic heuristics, improving their effectiveness. Well-known metaheuristics, such as Simulated Annealing, Tabu Search, GRASP, and Iterated Local Search (ILS), utilize stochastic local search (SLS) methods. These iterative processes usually yield feasible candidate solutions at each iteration and can be applied to many combinatorial optimization problems.

Data mining is crucial in extracting valuable knowledge from data [4, 15, 17, 19]. It involves automatically identifying patterns representing significant features and revealing concepts embedded in raw data. As a result, data mining becomes essential for informed decision-making in various contexts.

In [7], we presented a survey about incorporating data mining into hybrid metaheuristics to extract patterns from high-quality solution sets. The core idea is to use knowledge from previous heuristic iterations to guide future searches. It has been shown that heuristics can achieve a more effective exploration of the solution space by analyzing patterns derived from high-quality solutions.

This chapter updates the survey presented in [7] by highlighting recent advancements in applying data mining techniques within heuristics. In section “[Strategies to Use Data Mining in Heuristics](#),” we review strategies from the earlier survey that combine metaheuristics with data mining processes and introduce a new approach called MineReduce. Then, in section “[Hybrid Heuristics with Data Mining: A Selection of Cases](#),” we analyze various implementations across different applications and the corresponding computational results. Finally, section “[Conclusion](#)” offers concluding remarks and discusses potential future work in this evolving field.

Strategies to Use Data Mining in Heuristics

This section describes some strategies already presented in [7] to incorporate data mining in metaheuristics and a new approach called MineReduce.

DM-MSH and MDM-MSH Strategies

As described in [5], given a (combinatorial) problem Π , a stochastic local search algorithm for solving an arbitrary problem instance $\pi \in \Pi$ is defined by the following components:

- the search space $S(\pi)$ of instance π , which is a set of candidate solutions to π ;
- a set of (feasible) solutions $S'(\pi) \subseteq S(\pi)$;
- a neighborhood relation on $S(\pi)$, $N(\pi) \subseteq S(\pi) \times S(\pi)$;
- an initialization procedure $\text{INIT_PROC}(\pi)$ that draws an element from a probabilistic function $\text{init}(\pi)$, which specifies a probability distribution over initial search positions;
- a step procedure $\text{STEP_PROC}(\pi, s)$ that draws an element from a probabilistic function $\text{step}(\pi, s)$, which maps each given position s onto a probability distribution over its neighboring positions, for specifying the local search steps;
- a termination procedure $\text{TERM_PROC}(\pi, s)$ which uses a terminate predicate T that indicates the probability the search will be terminated upon reaching a specific point in the search space.

When using stochastic local search, the solutions obtained may differ in each algorithm run, so multistart SLS metaheuristics can find a different solution in each iteration.

So, the basic concept of incorporating a data mining process in multistart SLS metaheuristics is that patterns found in high-quality solutions obtained in earlier iterations can be used to conduct and improve the search process.

Two strategies to use data mining were presented in [7]. In the first, DM-MSH, the multistart heuristic (MSH) is divided into two phases. The first one is called *Elite Set Generation Phase*. It executes gen_elite_iter MSH iterations to obtain a set $\mathcal{S}$ of different solutions. The d best solutions from $\mathcal{S}$ compose an elite set $\mathcal{D}$.

Between the two phases, the data mining procedure is applied. It is responsible for extracting a set of patterns $\mathcal{P}$ from the elite set $\mathcal{D}$. The patterns to be mined are sets of elements frequently appearing in solutions in $\mathcal{D}$, which characterizes a frequent itemset mining application. A frequent itemset mined from $\mathcal{D}$ with support $s\%$ represents a set of elements that occur together in $s\%$ of the elite solutions.

Next, in the second phase, called *Hybrid Phase*, several slightly different MSH iterations are executed. The construction and local search heuristics may use a mined pattern $p \in \mathcal{P}$ in these iterations. For example, in constructive heuristics,

all elements of a pattern p may be inserted into the partial solution, from which a complete solution will be built by executing the standard construction procedure. This way, all constructed solutions will contain the elements of a pattern $p \in \mathcal{P}$. Likewise, local search procedures may use the elements of pattern p by requiring that only neighbors that contain elements from pattern p should be evaluated.

The pseudo-code of DM-MSH is illustrated in Fig. 1. In line 1, the elite set $\mathcal{D}$ is initialized with an empty set. The loop from line 3 to line 8 corresponds to the *Elite Set Generation Phase*, in which gen_elite_iter iterations of the multistart heuristic are performed. The original construction method is executed in line 4. Then, a local search method is performed in line 5. The elite set $\mathcal{D}$ is updated in line 6. In case of improvement, the best solution is updated in line 7. The data mining procedure extracts the set of patterns $\mathcal{P}$ from $\mathcal{D}$ in line 9. The loop from line 10 to line 15 corresponds to the *Hybrid Phase*. Each iteration is based on a pattern $p \in \mathcal{P}$, selected in line 11. The construction procedure that may use data mining is performed in line 12. In line 13, the local search that may use data mining is executed. In case of improvement, the best solution is updated in line 14. The best solution is returned in line 16.

The second strategy, called MDM-MSH, was developed to improve the results further. The main idea of this strategy is to mine more than once whenever the elite set is stable and good enough to be mined. The mining process will be executed: (a) as soon as the elite set first becomes stable, which means that no change in the elite set occurs throughout a given number of iterations, and (b) whenever the elite set has been changed and again has become stable.

Figure 2 shows the pseudo-code of MDM-MSH. The loop from line 4 to 10 corresponds to the first elite set generation phase, in which MSH iterations are performed until the elite set becomes ready to be mined or the total number of

Fig. 1 Pseudo-code of DM-MSH

```

begin DM_MSH( $n\_iter, \pi, f()$ )
1   $\mathcal{D} \leftarrow \emptyset$ ;
2   $f(\pi, best\_sol) \leftarrow \infty$ ;
3  for  $it = 1, \dots, gen\_elite\_iter$  do
4     $s \leftarrow CONSTRUCT\_SOL(\pi, f)$ ;
5     $s \leftarrow LOCAL\_SEARCH(s, \pi, f)$ ;
6     $UPDATE\_ELITE(\mathcal{D}, s)$ ;
7    if  $f(\pi, s) < f(\pi, best\_sol)$  then  $best\_sol \leftarrow s$ ;
8  end-for;
9   $\mathcal{P} \leftarrow MINE(\mathcal{D})$ ;
10 for  $it = gen\_elite\_iter + 1, \dots, n\_iter$  do
11    $p \leftarrow SELECT\_PATTERN(\mathcal{P})$ ;
12    $s \leftarrow DM\_CONSTRUCT\_SOL(p, \pi, f)$ ;
13    $s \leftarrow DM\_LOCAL\_SEARCH(p, s, \pi, f)$ ;
14   if  $f(\pi, s) < f(\pi, best\_sol)$  then  $best\_sol \leftarrow s$ ;
15 end-for;
16 return  $best\_sol$ ;
end DM_MSH.

```

Fig. 2 Pseudo-code of MDM-MSH

```

begin MDM_MSH( $n\_iter, \pi, f()$ )
1   $\mathcal{D} \leftarrow \emptyset$ ;
2   $f(\pi, best\_sol) \leftarrow \infty$ ;
3   $it \leftarrow 0$ ;
4  repeat
5     $s \leftarrow CONSTRUCT\_SOL(\pi, f)$ ;
6     $s \leftarrow LOCAL\_SEARCH(s, \pi, f)$ ;
7     $elite\_set\_is\_ready \leftarrow UPDATE\_ELITE(\mathcal{D}, s)$ ;
8    if  $f(\pi, s) < f(\pi, best\_sol)$  then  $best\_sol \leftarrow s$ ;
9     $it \leftarrow it + 1$ ;
10 until  $elite\_set\_is\_ready$  or  $it = n\_iter$ ;
11 while  $it < n\_iter$  do
12   if  $elite\_set\_is\_ready$  then  $\mathcal{D} \leftarrow MINE(\mathcal{D})$ ;
13    $p \leftarrow SELECT\_PATTERN(\mathcal{D})$ ;
14    $s \leftarrow DM\_CONSTRUCT\_SOL(p, \pi, f)$ ;
15    $s \leftarrow DM\_LOCAL\_SEARCH(p, s, \pi, f)$ ;
16    $elite\_set\_is\_ready \leftarrow UPDATE\_ELITE(\mathcal{D}, s)$ ;
17   if  $f(\pi, s) < f(\pi, best\_sol)$  then  $best\_sol \leftarrow s$ ;
18    $it \leftarrow it + 1$ ;
19 end-while;
20 return  $best\_sol$ ;
end MDM_MSH.

```

iterations is executed. Next, in the loop from line 11 to 19, the data mining procedure is performed in line 12 whenever the elite set is ready. In line 13, a pattern is selected. Then, the adapted construction is performed in line 14, and in line 15 the adapted local search is executed. The best solution is updated in line 17 if a better solution is found. After executing all iterations, the best solution is returned in line 20.

DM-MSH and MDM-MSH share a common approach. At some point, the algorithm benefits from knowledge accumulated in previous iterations. This approach relies on the assumption that there is a high probability that frequent patterns mined from a restricted set of high-quality solutions are part of other suboptimal and maybe even optimal solutions to the same problem instance. Thus, they may be inserted in initial solutions or required in neighbor solutions during a local search. The main difference between DM-MSH and MDM-MSH is when and how often patterns are mined.

According to Talbi's taxonomy for metaheuristics that incorporate machine learning (ML) [14], DM-MSH and MDM-MSH can be classified as *online low-level* ML-supported metaheuristics since they gather knowledge during the search while solving the problem (hence, online) and use that knowledge in processes that drive the initialization of solutions or local search (low-level components).

Experimental evidence shows that incorporating those strategies improved the effectiveness and efficiency of several heuristics for different combinatorial optimization problems, indicating that their underlying assumption holds.

MineReduce Strategy

Another approach named MineReduce was proposed based on the previous ideas. Since the mined patterns are assumed to likely be part of the best solutions to a problem instance, this approach temporarily fixes items from a pattern in the solution, equivalent to fixing (or at least constraining) values of decision variables related to those items. Therefore, the MineReduce approach applies a decomposition strategy that reduces the original problem instance to a subproblem by deleting or aggregating items based on the information obtained from a pattern. In essence, a pattern $p \in \mathcal{P}$ is selected for driving the following three-step process:

1. *Reduce*: A reduction operation is applied to the original problem instance $\pi \in \Pi$, removing or aggregating items from p (for being fixed in the solution), which results in a reduced instance $\pi' \in \Pi$;
2. *Optimize*: A solution $s' \in S'(\pi')$ – i.e., a feasible (and presumably good) solution to the reduced instance π' – is obtained through the application of some optimization method (e.g., SLS);
3. *Expand*: An expansion operation is applied to s' , reinserting or disaggregating the items from p , which results in a solution $s \in S'(\pi)$ – i.e., a feasible solution to the original instance π .

To picture this process more concretely, consider the classical 0-1 knapsack problem (01-KP), where we have a knapsack and a set of items, each with a weight and a value, and the objective is to fill the knapsack with the most valuable subset of items that it can accommodate. Specifically, consider the following toy instance of the 01-KP:

$$\pi = (W = 90, I = \{a, b, c, d, e, f, g, h, i, j\})$$

The knapsack in instance π has a capacity $W = 90$. It can hold items from the set $I = \{a, b, c, d, e, f, g, h, i, j\}$, whose properties are in Table 1.

Table 1 Items' properties for the 01-KP toy instance

Item	Weight	Value
<i>a</i>	9	50
<i>b</i>	13	35
<i>c</i>	18	20
<i>d</i>	38	16
<i>e</i>	15	60
<i>f</i>	68	45
<i>g</i>	27	60
<i>h</i>	39	40
<i>i</i>	23	30
<i>j</i>	52	10

Imagine we have designed and applied a problem-specific MSH for the 01-KP to our toy instance π . Suppose we have kept the best five solutions found across several iterations in an elite set $\mathcal{D} = \{s_1, s_2, s_3, s_4, s_5\}$, which are in Table 2.

Now, consider we have mined patterns from $\mathcal{D}$ with a minimum support of 60%. That means extracting the maximal frequent itemsets from $\mathcal{D}$, considering an itemset frequent if it is present in at least 60% of the solutions of $\mathcal{D}$. In this example, that would produce a pattern set $\mathcal{P} = \{p_1, p_2\}$. Those mined patterns are described in Table 3.

Once patterns have been mined, we can select one to drive MineReduce's reduce-optimize-expand process. For example, consider p_1 is selected. In the *reduce* step, we would assume that the items from p_1 must be part of a solution to π . Therefore, we would temporarily fix them in the solution and remove them from I , obtaining the following reduced instance:

$$\pi'_1 = (W'_1 = 53, I'_1 = \{c, d, f, g, h, i, j\})$$

In contrast, if pattern p_2 is selected instead of p_1 , then a different reduced instance is obtained:

$$\pi'_2 = (W'_2 = 39, I'_2 = \{b, c, d, f, h, i, j\})$$

Note that the knapsack capacity is considerably lower in the reduced instances (we subtract the pattern weights since its items are fixed in the solution), and so is the number of items one can pick to fill the knapsack. Hence, these instances have a much smaller solution space, which means they are much easier to solve than the original one.

Now, in the *optimize* step, consider that we applied an optimization method (e.g., SLS) to π'_1 and obtained the following solution: $s'_1 = \{g, i\}$ (with weight = 50 and value = 90). Finally, we must reverse the reduction operation in the *expand* step. That is, the items from p_1 (which had been fixed in the solution) must be inserted in s'_1 , which will produce a new solution to the original instance π : $s = \{a, b, e, g, i\}$ (with weight = 87 and value = 235).

Table 2 Elite solutions for the 01-KP toy instance

Solution	Weight	Value
$s_1 = \{a, b, e, h\}$	76	185
$s_2 = \{a, d, e, g\}$	89	186
$s_3 = \{a, b, c, e, i\}$	78	195
$s_4 = \{a, e, g, h\}$	90	210
$s_5 = \{a, b, c, e, g\}$	82	225

Table 3 Patterns mined for the 01-KP toy instance

Pattern	s_1	s_2	s_3	s_4	s_5
$p_1 = \{a, b, e\}$	✓		✓		✓
$p_2 = \{a, e, g\}$		✓		✓	✓

Fig. 3 Pseudo-code of MR-MSH

```

begin MR_MSH( $n\_iter, \pi, f()$ )
1   $\mathcal{D} \leftarrow \emptyset$ ;
2   $f(\pi, best\_sol) \leftarrow \infty$ ;
3   $it \leftarrow 0$ ;
4  repeat
5     $s \leftarrow \text{CONSTRUCT\_SOL}(\pi, f)$ ;
6     $s \leftarrow \text{LOCAL\_SEARCH}(s, \pi, f)$ ;
7     $elite\_set\_is\_ready \leftarrow \text{UPDATE\_ELITE}(\mathcal{D}, s)$ ;
8    if  $f(\pi, s) < f(\pi, best\_sol)$  then  $best\_sol \leftarrow s$ ;
9     $it \leftarrow it + 1$ ;
10 until  $elite\_set\_is\_ready$  or  $it = n\_iter$ ;
11 while  $it < n\_iter$  do
12   if  $elite\_set\_is\_ready$  then  $\mathcal{D} \leftarrow \text{MINE}(\mathcal{D})$ ;
13    $p \leftarrow \text{SELECT\_PATTERN}(\mathcal{D})$ ;
14    $\pi' \leftarrow \text{REDUCE}(\pi, p)$ ;
15    $s' \leftarrow \text{CONSTRUCT\_SOL}(\pi', f)$ ;
16    $s' \leftarrow \text{LOCAL\_SEARCH}(s', \pi', f)$ ;
17    $s \leftarrow \text{EXPAND}(s', p)$ ;
18    $s \leftarrow \text{LOCAL\_SEARCH}(s, \pi, f)$ ;
19    $elite\_set\_is\_ready \leftarrow \text{UPDATE\_ELITE}(\mathcal{D}, s)$ ;
20   if  $f(\pi, s) < f(\pi, best\_sol)$  then  $best\_sol \leftarrow s$ ;
21    $it \leftarrow it + 1$ ;
22 end-while;
23 return  $best\_sol$ ;
end MR_MSH.

```

Since MDM-MSH has generally performed better than DM-MSH, its adaptive multi-mining structure has been adopted in the MineReduce-based MSH. Figure 3 shows the pseudo-code of MR-MSH. It has essentially the same structure as MDM-MSH. The differences can be observed from line 14 to line 18. In line 14, the problem instance π is reduced to π' based on the selected pattern p . Lines 15 and 16 correspond to a typical implementation of the *Optimize* step from the MineReduce approach, consisting of one MSH iteration (construction and local search) on $S(\pi')$, which produces a locally optimal solution s' to the reduced instance π' . In line 17, s' is expanded into the corresponding solution s to instance π based on pattern p . Finally, in line 18, s is the initial solution for a local search on $S(\pi)$.

According to Talbi's taxonomy [14], like DM-MSH and MDM-MSH, MR-MSH can be classified as an *online low-level* ML-supported metaheuristic. However, it is additionally (and primarily) classified as a *problem-level* ML-supported metaheuristic since it uses the discovered knowledge for hierarchical problem decomposition (defining and solving smaller subproblems).

The following section presents several cases showing that incorporating data mining using the above strategies improved the base heuristics' solution quality and convergence speed.

Hybrid Heuristics with Data Mining: A Selection of Cases

This section presents a selection of cases where data mining has been incorporated into heuristics for different combinatorial optimization problems.

In [7], several examples show the integration of data mining with heuristics based on Greedy Randomized Adaptive Search Procedures (GRASP) applied to problems with different characteristics. The first was proposed to solve the server replication for reliable multicast problem. A solution to this problem is an unordered set of elements. The second GRASP was proposed to solve the traveling salesperson problem with pickup and delivery. This problem has solutions that are ordered sets of elements, which require a different approach. In the last two of these four cases, data mining was integrated with hybrid versions of GRASP that include a path-relinking strategy, one for the two-path network design problem and another for the p -median problem. Data mining was also integrated with heuristics based on Iterated Local Search (ILS) to solve the set covering with pairs problem in [7].

Section “[DM-Supported HGS for the Capacitated VRP](#)” shows the results obtained by incorporating the MDM-MSH strategy into a Hybrid Genetic Search (HGS) heuristic for the capacitated vehicle routing problem.

Then, in section “[DM-Supported MS-ILS for the Heterogeneous Fleet VRP](#),” three multistart versions of ILS using data mining strategies (respectively DM-MSH, MDM-MSH, and MR-MSH) are presented and evaluated for the heterogeneous fleet vehicle routing problem.

Finally, in section “[DM-Supported MS-ILS for the Multisource Capacitated FLP with Customer Incompatibilities](#),” the MR-MSH approach is combined with the ILS heuristic for the multisource capacitated facility location problem with customer incompatibilities.

The computational results from all these applications confirm that frequent patterns mined from restricted sets of high-quality solutions can improve the performance of heuristic methods regarding solution quality and convergence speed.

DM-Supported HGS for the Capacitated VRP

This section summarizes the results obtained by incorporating data mining into a multistart SLS heuristic for the capacitated vehicle routing problem (CVRP). An application of the MDM-MSH strategy was proposed for this problem [10] and ranked 2nd in the CVRP track of the 12th DIMACS Implementation Challenge (<http://dimacs.rutgers.edu/programs/challenge/vrp/cvrp>).

Problem Definition

The CVRP is described as follows. Let $G = (V, A)$ be a directed graph, where $V = \{0, 1, \dots, n-1\}$ is a set composed of n vertices and $A = \{(i, j) : i, j \in V, i \neq j\}$ is the set of arcs. Vertex 0 is the depot from where the vehicles depart, whereas the

set $V' = V \setminus \{0\}$ consists of the remaining vertices representing $n - 1$ customers. Each customer $i \in V'$ is associated with a nonnegative quantity q_i that specifies the demand for some resource, and a vehicle can carry a maximum quantity Q of the resource. Each arc $(i, j) \in A$ has an associated cost d_{ij} representing the distance between vertices i and j . A route is defined by a sequence of vertices $R = (i_1, i_2, \dots, i_{|R|})$, $i_1 = i_{|R|} = 0$, and $\{i_2, \dots, i_{|R|-1}\} \subseteq V'$; that is, each route is a circuit in G that starts and ends at the depot. A route R is feasible if the sum of all customers' demands on R does not exceed the vehicle capacity Q . A route's cost is the sum of the costs (distances) associated with each traversed arc. The aim is to discover feasible routes such that each customer is visited precisely once, and the sum of all route costs is minimized. There are no restrictions on the number of routes in a solution.

Data Mining Application

The proposed method was built by incorporating data mining into a state-of-the-art Hybrid Genetic Search (HGS) metaheuristic for the CVRP [16]. HGS is an iterative population-based metaheuristic that restarts the population whenever it completes a number of consecutive iterations without improving the overall best solution.

In MDM-HGS, the elite set E is updated whenever a new feasible solution is inserted in the population if E is not full or the new solution is better than the worst solution in E . It is considered *stable* when one of the following criteria is met: (i) E has not been mined yet, and its contents have not changed for a number of consecutive restarts greater than or equal to δR_{MAX} , where δ is a parameter and R_{MAX} is the maximum number of restarts, dynamically estimated based on the time limit, the elapsed time, and the number of completed restarts; or (ii) the contents of E have changed after it was last mined, but they have not changed for a number of consecutive restarts greater than or equal to δR_{MAX} .

The data mining method returns frequent patterns found in E . Each pattern is a set of paths connecting customers. Figure 4 illustrates this data mining process with four distinct solutions for a CVRP instance. Suppose these solutions compose an elite set for the corresponding instance and let the minimum support be 100%. In that case, the data mining method would extract a pattern composed of the paths that appear in all four solutions (thick lines in the figure).

A new solution generation method is introduced in MDM-HGS. It is based on a randomized version of the Clarke and Wright (CW) heuristic and uses the paths from a mined pattern to initialize routes. Its randomization mechanism is a roulette wheel selection as in the CW implementation discussed in [13]. The *savings list* S is built when the problem instance data is loaded. It contains all edges that connect two customers in the input graph, sorted in decreasing order of their corresponding savings values. A savings value represents the cost savings obtained by concatenating two routes, one where customer i is adjacent to the depot and another where customer j is adjacent to the depot, via the insertion of the edge $\langle i, j \rangle$ in the solution. It is given by $s_{ij} = d_{0i} + d_{0j} - d_{ij}$, where d_{ij} is the distance between vertices i and j , and vertex 0 is the depot.

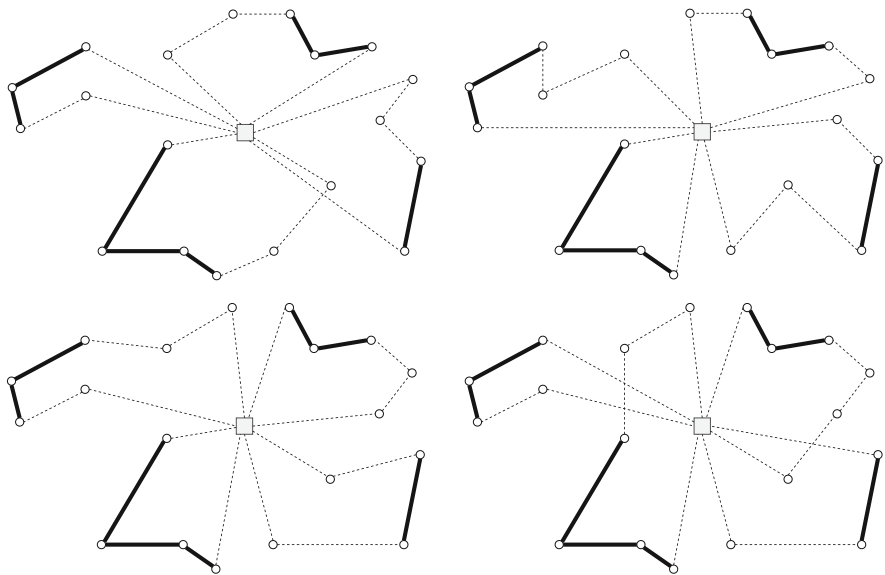


Fig. 4 Four distinct solutions for a CVRP instance and a pattern with 100% support. (Source: [10, p. 5])

The randomized CW solution generation based on a pattern p is described in Fig. 5. The solution is initialized with one route for each path in p (lines 1–3) and one for each customer not in p (lines 4–6). Then, the savings list S is iteratively processed (lines 7–13). At each step, an edge $\langle i, j \rangle$ is selected among the t (a random number between 2 and 6, inclusive) first unvisited entries in S using the roulette wheel method (line 9). Suppose customers i and j are adjacent to the depot on two different routes, and the vehicle capacity is sufficient for the total demand of both routes. In that case, these routes are concatenated by inserting $\langle i, j \rangle$ in the solution (lines 10–12). Finally, after S has been processed, a greedy strategy is used to avoid an excessive number of routes: for each route with only one customer i , i is moved to the end of a route with more than one customer. Each move is the cheapest, feasible if possible (line 16), infeasible otherwise (line 18).

Computational Results

Extensive computational experiments were conducted to compare MDM-HGS and the original method HGS-CVRP concerning the best solution found (within a given time-out) and the primal integral (PI) [1], which aims to measure the progress of the primal bound's convergence toward the optimal solution over the entire solving time. The smaller it is, the better the expected quality of the incumbent solution if the solving procedure is stopped at an arbitrary point in time. For each combination of instance and algorithm, the following values were registered: the average solution

```

begin DM_RCW( $p$ )
1  for each path in  $p$  do
2    Initialize a route with the current path;
3  end-for;
4  for each customer not in  $p$  do
5    Initialize a route with the current customer;
6  end-for;
7  while  $S$  contains unvisited positive-valued entries do
8     $t \leftarrow \text{RAND}(2, 6)$ ;
9     $\langle i, j \rangle \leftarrow \text{ROULETTE\_WHEEL}(S, t)$ ;
10   if two routes can be feasibly concatenated through  $\langle i, j \rangle$  then
11     Concatenate the two routes through  $\langle i, j \rangle$ ;
12   end-if;
13 end-while;
14 for each route with only one customer  $i$  do
15   if  $i$  can be feasibly moved to a route with more than one customer then
16     Make the cheapest feasible move of  $i$  to the end of another route;
17   else
18     Make the cheapest infeasible move of  $i$  to the end of another route;
19   end-if;
20 end-for;
21 return the generated solution;
end DM_RCW.

```

Fig. 5 Pseudo-code of the pattern-based randomized CW solution generation

cost (Avg Cost), the percentage gap of Avg Cost to the BKS (Gap), the best solution cost (Best Cost), and the average primal integral (Avg PI).

Table 4 summarizes the results, showing for each algorithm the following: the average Gap (Avg Gap); the global average PI; the number of wins in Avg Cost, Best Cost, and Avg PI; and the number of best-known solutions found. Improving values are highlighted in bold. This comparison shows that MDM-HGS obtains solutions of higher quality and better PI values than HGS-CVRP in most instances. The only exception is the DIMACS subset, where HGS-CVRP has a slightly lower PI, but that difference is not statistically significant. On the other hand, MDM-HGS has a statistically significant advantage in the average gap for the same subset.

The 12th DIMACS Implementation Challenge had 16 teams competing in the CVRP track. All competing algorithms were ranked according to their PI value for each instance. The best algorithm got 10 points, the second 8, then 6, 5, 4, 3, 2, and 1. The final ranking is based on the total point score obtained by each method in all instances. Table 5 presents the rankings from Phase 1 for all instances and the five subsets separately.

The Phase 1 results show that MDM-HGS was very competitive. It ranked first in 2 of the 5 instance subsets: X, the largest one with 100 instances, and Classic, for which MDM-HGS achieved an almost perfect score with 66 points of the 70 possible. Besides, despite ranking fourth in the Golden subset, that was a good performance, considering that the margin among the top-four algorithms in this

Table 4 Summarized comparison of HGS-CVRP to MDM-HGS

	All (141)		X (100)		Belgium (10)		Classic (7)		Golden (12)		DIMACS (12)	
	HGS	MDM	HGS	MDM	HGS	MDM	HGS	MDM	HGS	MDM	HGS	MDM
Avg Gap	0.40%	0.33% ^a	0.08%	0.07% ^a	4.74%	3.75% ^a	0.00%	0.00%	0.07%	0.07%	0.09%	0.08% ^a
Global Avg PI	0.4884	0.3981 ^a	0.1176	0.1145	5.3981	4.1644 ^a	0.0047	0.0044	0.1190	0.1124	0.1384	0.1387
Wins Avg Cost	34	61 ^a	24	40 ^a	1	9 ^a	1	1	4	4	4	7
Wins Best Cost	21	47 ^a	14	32 ^a	1	9 ^a	1	0	2	2	3	4
Wins Avg PI	53	81 ^a	41	54	0	10 ^a	2	3	5	7	5	7
Nb BKS	68	72	50	53	0	0	7	6	7	8	4	5

^aStatistically significant advantage

Table 5 Rankings from the 12th DIMACS Implementation Challenge – CVRP Track – Phase 1

All (141)		X (100)		Belgium (10)		Classic (7)		Golden (12)		DIMACS (12)	
Solver	Score	Solver	Score	Solver	Score	Solver	Score	Solver	Score	Solver	Score
FHCSolver	931	MDM-HGS	663	FSP4D	100	MDM-HGS	66	FHCSolver	86	FHCSolver	83
MDM-HGS	896	FHCSolver	642	FHCSolver	80	POP-HGS	51	POP-HGS	83	FSP4D	75
POP-HGS	814	POP-HGS	605	AILS-II	56	FHCSolver	40	FSP4D	81	POP-HGS	74
FSP4D	737	HGSRR	480	HGSRR	44	FSP4D	34	MDM-HGS	80	HGSRR	73
HGSRR	678	FSP4D	447	LKHSP	36	HGSRR	28	HGSRR	53	MDM-HGS	62
AILS-II	558	AILS-II	416	MDM-HGS	25	MAESN	19	MAESN	35	AILS-II	52
MAESN	406	MAESN	352	optaplanner	19	ILS-SP	12	AILS-II	25	GA+TBD	29
GA+TBD	291	GA+TBD	222	ALNS++	17	AILS-II	9	GA+TBD	25	LKHSP	8
ILS-SP	70	ILS-SP	49	GA+TBD	7	GA+TBD	8	LKHSP	0	ALNS++	8
LKHSP	57	ALNS++	17	ILS-SP	5	LKHSP	6	ALNS++	0	ILS-SP	4
ALNS++	42	LKHSP	7	POP-HGS	1	optaplanner	0	ILS-SP	0	optaplanner	0
optaplanner	19	optaplanner	0	MAESN	0	ALNS++	0	optaplanner	0	Clowder	0
Clowder	0	Clowder	0	Clowder	0	Clowder	0	Clowder	0	MAESN	0
Resilience	0	Resilience	0	Resilience	0	OR-Tools	0	OR-Tools	0	Resilience	0
DDVRP	0	DDVRP	0	OR-Tools	0	Resilience	0	Resilience	0	OR-Tools	0
OR-Tools	0	OR-Tools	0	DDVRP	0	DDVRP	0	DDVRP	0	DDVRP	0

subset was very tight (only six points). The results obtained in the remaining two subsets were also above average, ranking fifth and sixth, respectively.

The top 8 competitors from Phase 1 qualified for the second and final phase, in which 100 new (undisclosed) instances were used. Table 6 presents the rankings from Phase 2 for all 100 instances and partitions concerning customer positioning and average route size.

Phase 2 results confirmed that MDM-HGS is very competitive. The instance partitioning reveals that it outperforms all competitors for two classes and ties in first for another. Regarding customer positioning, the instances are partitioned into three classes: “clustered,” “random,” and “random-clustered” (half of the customers are clustered, and the remaining customers are randomly positioned). MDM-HGS ranked first in the clustered instances. Naturally, these instances are more prone to the occurrence of patterns than random ones, favoring the pattern-based strategy applied in the MDM-MSH framework. Nevertheless, MDM-HGS also achieved high scores for the other partitions, tying for first place with FHCSolver in the random instances and ranking second in the random-clustered ones.

The average route size is defined as the number of customers divided by the minimum number of vehicles. Regarding this aspect, the instances are partitioned into “long-route” (the 50 with longest routes) and “short-route” (the 50 with shortest routes). MDM-HGS ranked first in the long-route instances. The longer the routes in solutions to an instance, the more likely long paths will occur in patterns mined from those solutions. Hence, long-route instances seem to favor the pattern-based strategy applied in the MDM approach more than short-route ones.

MDM-HGS was a close second to FHCSolver in the overall ranking. One interesting observation is that FHCSolver combines the original HGS-CVRP with an adapted version of another algorithm (FILO). Its strategy relies primarily on selecting which algorithm to use based on the instance’s number of customers and average route size. Since the experimental results provide evidence that MDM-HGS outperforms HGS-CVRP, ultimately, FHCSolver’s performance could probably be improved by simply replacing HGS-CVRP with MDM-HGS within its framework.

These results show that MDM-HGS is a competitive method in the CVRP state of the art, overcoming the original HGS-CVRP and many other novel algorithms. MDM-HGS was the first heuristic to apply this pattern-based approach in a population-based algorithm, which depicts an evolutionary framework, showing that data mining can improve performance in such a context.

DM-Supported MS-ILS for the Heterogeneous Fleet VRP

This section summarizes the results obtained by incorporating data mining into a multistart SLS heuristic for the heterogeneous fleet vehicle routing problem (HFVRP). Applications of the DM-MSH and MDM-MSH strategies were proposed for this problem in [8], and an application of the MR-MSH strategy was proposed in [9], the very first application of the MineReduce approach.

Table 6 Rankings from the 12th DIMACS Implementation Challenge – CVRP Track – Phase 2

All (100)		Clustered (32)		Random (34)		Rd-clustered (34)		Long-route (50)		Short-route (50)	
Solver	Score	Solver	Score	Solver	Score	Solver	Score	Solver	Score	Solver	Score
FHCSolver	643	MDM-HGS	213	FHCSolver	217	FHCSolver	236	MDM-HGS	334	FHCSolver	329
MDM-HGS	622	FHCSolver	190	MDM-HGS	217	MDM-HGS	192	FHCSolver	314	AILS-II	308
AILS-II	536	POP-HGS	184	AILS-II	184	AILS-II	179	FSP4D	274	MDM-HGS	288
POP-HGS	525	AILS-II	173	HGSRR	177	FSP4D	175	POP-HGS	272	POP-HGS	253
HGSRR	477	FSP4D	136	POP-HGS	175	HGSRR	166	HGSRR	236	HGSRR	241
FSP4D	458	HGSRR	134	FSP4D	147	POP-HGS	166	AILS-II	228	MAESN	203
MAESN	389	MAESN	132	MAESN	132	MAESN	125	MAESN	186	FSP4D	184
GA+TBD	250	GA+TBD	86	GA+TBD	77	GA+TBD	87	GA+TBD	106	GA+TBD	144

Problem Definition

The HFVRP is described as follows. Let $G = (V, A)$ be a directed graph, where $V = \{0, 1, \dots, n\}$ is a set composed of $n + 1$ vertices and $A = \{(i, j) : i, j \in V, i \neq j\}$ is the set of arcs. Vertex 0 is the depot, where the vehicle fleet is located, whereas the set $V' = V \setminus \{0\}$ consists of the remaining vertices representing the n customers. Each customer $i \in V'$ is associated with a nonnegative demand q_i . The fleet comprises m distinct vehicle types, which compose a set $M = \{1, \dots, m\}$. For each vehicle type $u \in M$, there are m_u vehicles available, each with a capacity Q_u and a fixed cost f_u . Finally, for each combination of an arc $(i, j) \in A$ and a vehicle type $u \in M$, there is an associated cost $c_{ij}^u = d_{ij}r_u$, where d_{ij} is the distance between vertices i and j , and r_u is the dependent (variable) cost per unit distance associated with the vehicle type u .

A route is defined by a pair (R, u) , where $R = (i_1, i_2, \dots, i_{|R|})$, $i_1 = i_{|R|} = 0$, and $\{i_2, \dots, i_{|R|-1}\} \subseteq V'$; that is, each route is a circuit in G that starts and ends at the depot and is assigned to a vehicle of type $u \in M$. A route (R, u) is feasible if the sum of all customers' demands on R does not exceed the capacity Q_u of the vehicle assigned to it. A route's cost is the sum of the assigned vehicle's fixed cost and the dependent costs associated with each traversed arc and the vehicle type. Thus, the aim is to discover feasible routes such that each customer is visited precisely once, the total quantity of routes assigned to vehicles of each type $u \in M$ does not exceed m_u , and the sum of all route costs is minimized.

Data Mining Application

The proposed methods were built by incorporating data mining into a multistart ILS (MS-ILS) heuristic for the HFVRP [12], based on the DM-MSH, MDM-MSH, and MR-MSH strategies. They are called DM-MS-ILS, MDM-MS-ILS, and MR-MS-ILS, respectively.

In these proposals, solutions in the elite set are represented as sets of pairs containing an arc and a vehicle type. This representation allows the application of the data mining procedure to extract maximal frequent itemsets. For each route (R, u) , the list R is decomposed into a set of arcs $\{(i_1, i_2), (i_2, i_3), \dots, (i_{|R|-1}, i_{|R|})\}$. Then, the vehicle type u is assigned to each arc in the set, resulting in a set in which each element is a pair composed of an arc (i_r, i_{r+1}) , $r = 1, 2, \dots, |R| - 1$, in R and the vehicle type u , with the form $\{((i_1, i_2), u), ((i_2, i_3), u), \dots, ((i_{|R|-1}, i_{|R|}), u)\}$. An arc must be present and associated with the same vehicle type in a minimum number of solutions in the elite set to belong to a pattern. Consequently, the patterns mined are formed of route segments, each with a vehicle type assigned.

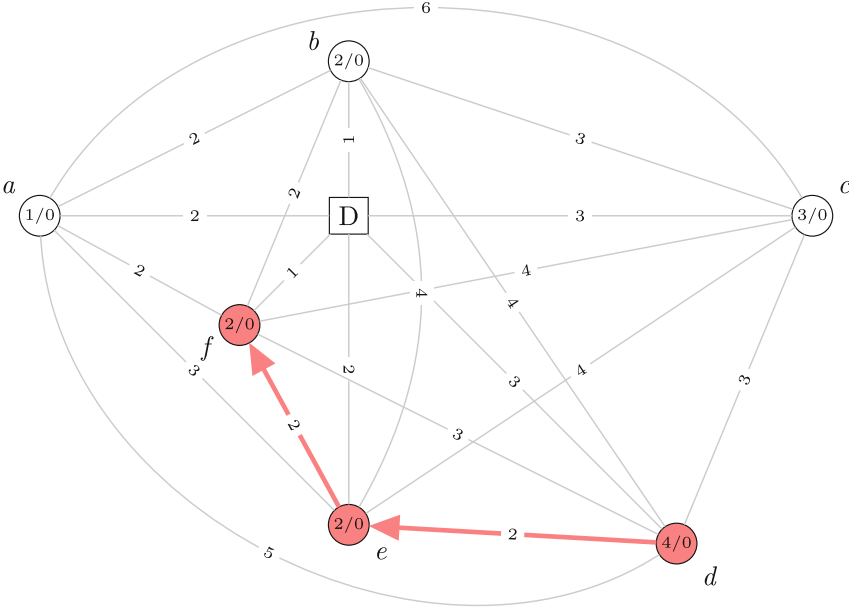
In MR-MS-ILS, the *Reduce* step of MineReduce relies on the use of patterns mined from the elite set to perform a vertex-based reduction procedure by merging customer vertices that appear consecutively (in the same route segment) in a pattern into one customer cluster vertex.

It is necessary to extend the HFVRP model to use this strategy. Since it must be possible to represent a route segment as a customer vertex, each customer $i \in V$ must have an associated l_i value corresponding to the inner length of the underlying

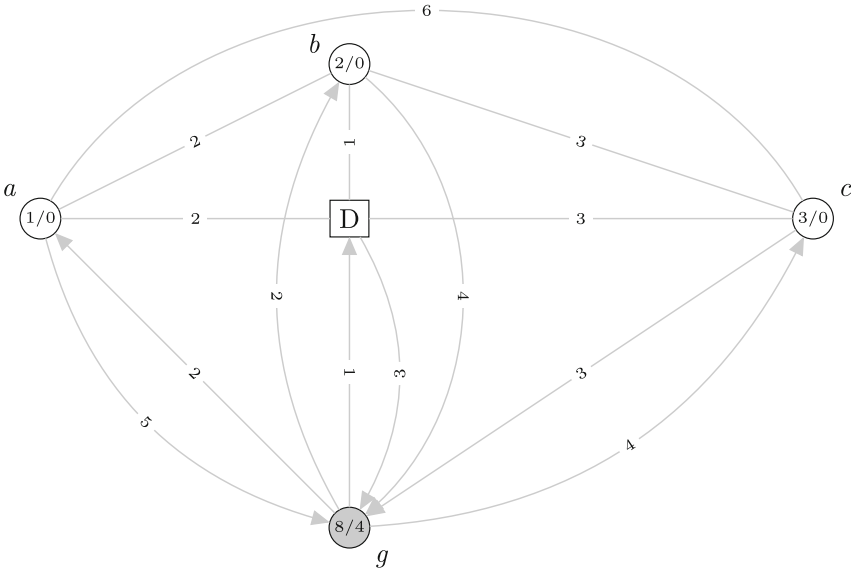
route segment. This value is used to calculate the cost $c_i^u = l_i r_u$, which represents the variable cost for a vehicle of type u to traverse the route segment represented by i , for each combination of a customer $i \in V$ and a vehicle type $u \in M$. Customer vertices of regular (non-reduced) instances and those of reduced instances that are not customer cluster vertices have an inner length of 0. The cost of a route becomes, in this extended model, the sum of the fixed cost of the vehicle associated with the route and the variable costs related to (i) the combination of the vehicle type and each traversed arc and (ii) the combination of the vehicle type and each visited customer vertex.

Let $G = (V, A)$ be a directed graph associated with an HFVRP instance and p a pattern consisting of a set of route segments in that instance, each segment defined by a pair (R', u) , where $R' = (i_1, i_2, \dots, i_{|R'|})$. Let $G^* = (V^*, A^*)$ be a directed graph associated with a reduced version of the instance associated with G based on p . Such a reduced version can be obtained as follows. Initially, G^* is defined as a copy of G . For each route segment $(R', u) \in p$, each of the customers in R' is removed from G^* – that is, the vertex corresponding to the customer is removed from V^* and the arcs that connect that vertex to the others are removed from A^* . Also, a customer cluster vertex $i_{R'}$ corresponding to the route segment is added to V^* and arcs connecting $i_{R'}$ to the other vertices in V^* are added to A^* . The demand for $i_{R'}$ is given by $q_{i_{R'}} = \sum_{r'=1}^{|R'|} q_{i_{r'}}$, that is, the sum of customer demands in R' . The distance from each vertex $i^* \in V^*$ to $i_{R'}$ is given by $d_{i^* i_{R'}} = d_{i^* i_1}$, that is, the distance from i^* to i_1 (the first customer in R'). The distance from $i_{R'}$ to each vertex $i^* \in V^*$ is given by $d_{i_{R'} i^*} = d_{i_{|R'|} i^*}$, that is, the distance from $i_{|R'|}$ (the last customer in R') to i^* . Finally, the inner length of $i_{R'}$ is given by $l_{i_{R'}} = \sum_{r=1}^{|R'|-1} d_{i_{r'} i_{r'+1}}$, that is, the sum of the distances between consecutive customers in R' .

Figure 6a presents a representation of a graph $G = (V, A)$ in the extended model described above. In this representation, the customer vertices present, in addition to the value of the demand, the value of the inner length. In this non-reduced instance, all customers have inner length 0. The pattern used in the presented reduction is highlighted in the graph. It consists of a single route segment: (R'_1, u_1) , where $R'_1 = (d, e, f)$. The graph $G^* = (V^*, A^*)$ associated with the reduced version of the instance is presented in Fig. 6b. The customer vertices in R'_1 are replaced in the graph by a customer cluster represented by vertex g . This customer cluster's demand q_g is given by the sum of the customers' demands it replaces: $q_g = q_d + q_e + q_f = 4 + 2 + 2 = 8$. The distance from each of the other vertices to g is given by the distances from these vertices to d (first customer in R'_1): $d_{Dg} = d_{Dd} = 3$, $d_{ag} = d_{ad} = 5$, $d_{bg} = d_{bd} = 4$, and $d_{cg} = d_{cd} = 3$. The distance from g to each of the other vertices is given by the distances from f (last customer in R'_1) to these vertices: $d_{gD} = d_{fD} = 1$, $d_{ga} = d_{fa} = 2$, $d_{gb} = d_{fb} = 2$, and $d_{gc} = d_{fc} = 4$. Finally, the inner length l_g of g is given by the sum of the distances between consecutive customers in R'_1 : $l_g = d_{de} + d_{ef} = 2 + 2 = 4$.



(a) Non-reduced instance (with a mined pattern highlighted)



(b) Reduced instance

Fig. 6 An example of reduction of an HFVRP instance based on a mined pattern. (Source: [9, p. 6]). (a) Non-reduced instance (with a mined pattern highlighted). (b) Reduced instance

Computational Results

Extensive computational experiments were conducted to compare the original MS-ILS and its hybrid versions with data mining. The first experimental comparison [8] showed that both DM-MS-ILS and MDM-MS-ILS significantly outperformed the original MS-ILS, with MDM-MS-ILS being the most effective and efficient of those two methods with data mining. The most recent computational experiments [9] compared MS-ILS, MDM-MS-ILS, and MR-MS-ILS.

The problem instances used in the computational experiments were the HFVRP benchmark instances proposed in [2], divided into 4 sets: Set 1, with 15 instances having fewer than 100 customers; Set 2, with 38 instances ranging from 100 to 150 customers; Set 3, with 31 instances ranging from 151 to 200 customers; and Set 4, with 12 instances having more than 200 customers. One instance from each set was reserved for parameter tuning and not used in the comparison.

Table 7 summarizes the results from this comparison, presenting the numbers of wins/ties (the number of instances for which a method achieved the best results) and the average percentage difference (APD) to the original MS-ILS for each instance set separately and for all instances (global).

The results show a clear superiority of MR-MS-ILS to the other two methods. It achieved better solution quality for most instances and much faster convergence for all of them, as it executes the same number of iterations in less computational time. MR-MS-ILS attained better average costs for 83% of the instances (65% with statistical significance) compared to the MS-ILS and better average costs for 76% of the instances (64% with statistical significance) in comparison with the MDM-MS-ILS. If we exclude the small instances (Set 1) where the three heuristics have shown similar performance, the superiority of MR-MS-ILS becomes more evident. It has better average costs for 91% of the instances (76% with statistical significance) compared to MS-ILS and better average costs for 86% of the instances (74% with statistical significance) compared to MDM-MS-ILS.

Additional analyses were performed to inspect the behavior of the methods further. Figure 7 evaluates the solution costs obtained throughout one execution of each method. The charts in the first row present, for each method, the solution costs obtained per iteration in the generation and local search phases, whereas the second row provides enlarged views focusing on the local search phase curve. The charts in Fig. 8 exhibit the computational time spent in the generation and local search phases per iteration. The dashed vertical lines indicate the iterations preceding a data mining method run.

In Fig. 7, the charts in the first row show that the reduction in the costs of the solutions generated by MR-MS-ILS after data mining is much more expressive than that observed for MDM-MS-ILS. The enlarged views in the second row show that the costs of generated solutions fall to a level even lower than that of the costs of solutions discovered through the local search in previous iterations, and the local search finds even better solutions after each run of the data mining procedure.

On the other hand, as shown in the last chart of Fig. 8, the computational time spent in the generation phase – which is close to zero in the initial iterations –

Table 7 Summarized results comparing MS-ILS, MDM-MS-ILS, and MR-MS-ILS

		MS-ILS			MDM-MS-ILS			MR-MS-ILS		
		Best Cost	Avg. Cost	Avg. Time	Best Cost	Avg. Cost	Avg. Time	Best Cost	Avg. Cost	Avg. Time
Set 1	Wins/Ties	10	7	–	13	7	–	13	6	14
	APD				– 0.01%	0.00%	– 14.28%	– 0.01%	0.05%	– 63.72%
Set 2	Wins/Ties	12	3	–	11	8	–	29	26	37
	APD				– 0.01%	– 0.04%	– 13.71%	– 0.15%	– 0.19%	– 65.09%
Set 3	Wins/Ties	1	–	–	1	–	–	29	30	30
	APD				– 0.06%	– 0.01%	– 13.83%	– 0.39%	– 0.46%	– 57.01%
Set 4	Wins/Ties	–	–	–	–	–	–	11	11	11
	APD				– 0.10%	– 0.03%	– 14.34%	– 0.56%	– 0.52%	– 53.24%
Global	Wins/Ties	23	10	–	25	15	–	82	73	92
	APD				– 0.04%	– 0.02%	– 13.91%	– 0.26%	– 0.28%	– 60.83%

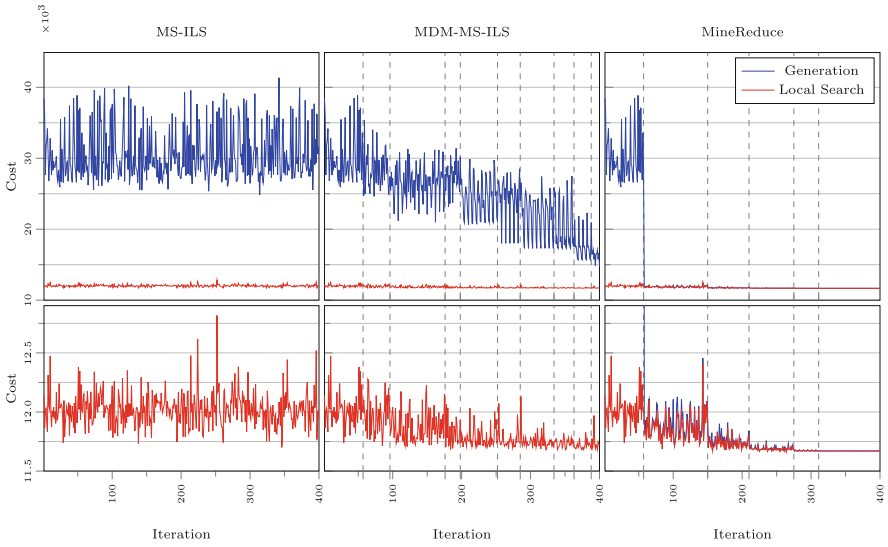


Fig. 7 Cost vs. iteration charts illustrating the behavior of MS-ILS, MDM-MS-ILS, and MR-MS-ILS (MineReduce). (Source: [9, p. 11])

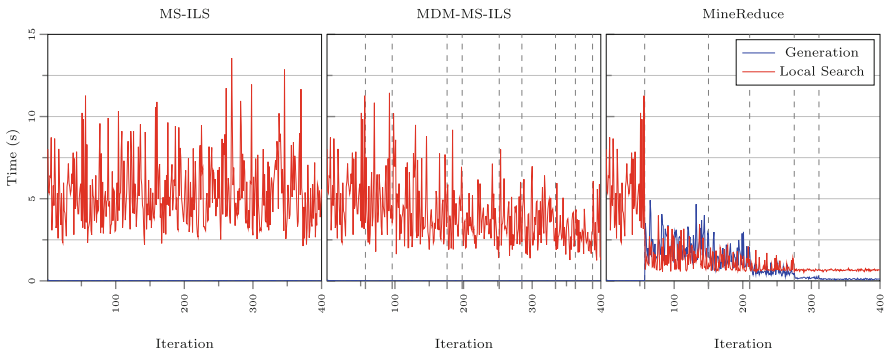


Fig. 8 Time vs. iteration charts illustrating the behavior of MS-ILS, MDM-MS-ILS, and MR-MS-ILS (MineReduce). (Source: [9, p. 11])

increases after the first run of the data mining procedure. This increase is due to the execution of the problem size reduction process, which includes a local search on a reduced version of the problem instance. However, the increase in time spent in the generation phase is compensated by an expressive reduction in time spent in the local search phase. After data mining, the combined time spent in the generation and local search phases per iteration is significantly reduced.

The overall reduction in computational time can be explained by the fact that MineReduce shrinks the problem instance and, consequently, the search space. Therefore, the first local search (embedded in MineReduce’s generation phase over

the reduced instance) performs a much smaller number of movement evaluations. Afterward, the second local search (actual local search phase, over the original instance) starts from a higher-quality solution, converging faster.

In MR-MS-ILS, as the charts show, the reduction of solution costs and computational time is intensified after each execution of the data mining procedure, reaching very low levels in the last iterations, which explains the superiority demonstrated by the results of the experiments presented in Table 7.

One last experiment was conducted to assess convergence speed based on a time-to-target (TTT) plot. The target was a solution with a cost lower than or equal to the average cost obtained by MS-ILS in the experiment summarized in Table 7. The obtained chart is shown in Fig. 9.

The chart shows that MR-MS-ILS converges faster than the other methods. It is possible to observe that the probability that the target will be reached within 200 seconds, for example, is approximately 97% for MR-MS-ILS, 76% for MDM-MS-ILS, and 67% for MS-ILS.

The analyses of the charts presented in Figs. 7, 8, and 9 clearly illustrate the influence of the MineReduce approach in the behavior of metaheuristics. As expected, this behavior improves solution quality and convergence speedup.

DM-Supported MS-ILS for the Multisource Capacitated FLP with Customer Incompatibilities

This section summarizes the results obtained by incorporating data mining into a multistart SLS heuristic for the Multisource Capacitated Facility Location Problem with Customer Incompatibilities (MS-CFLP-CI). The MS-CFLP-CI is a recently

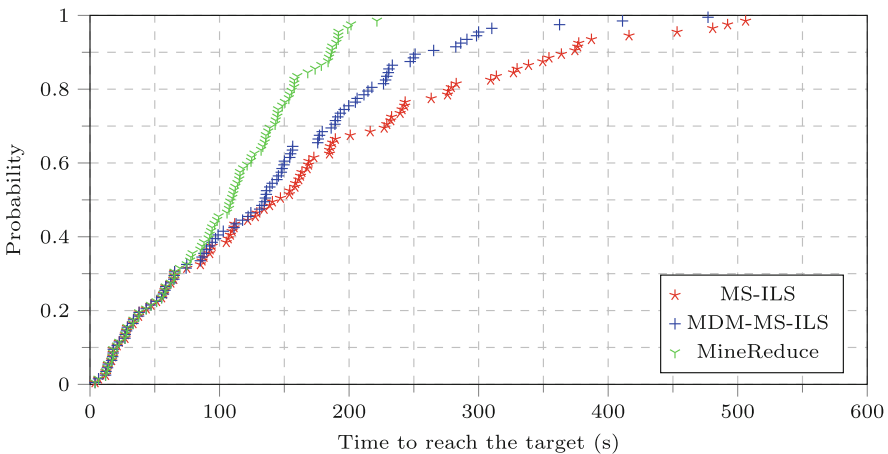


Fig. 9 TTT plot comparing MS-ILS, MDM-MS-ILS, and MR-MS-ILS (MineReduce). (Source: [9, p. 12])

introduced variant of the capacitated facility location problem, proposed in the MESS 2020+1 Metaheuristics Competition (<https://www.ants-lab.it/mess2020/#competition>). A MineReduced-based multistart ILS (MR-MS-ILS) heuristic was proposed for this problem and ranked first in the competition [11].

Problem Definition

Let $\mathcal{J} = \{1, \dots, J\}$ be a set of facilities such that each facility $j \in \mathcal{J}$ has a capacity s_j and an opening cost f_j , and let $\mathcal{I} = \{1, \dots, I\}$ be a set of customers such that each customer i has a demand of quantity of goods d_i to be completely satisfied by one or more facilities. The shipping cost c_{ij} is the cost per unit of transporting goods from facility j to customer i . Finally, a set Γ of pairs of incompatible customers is defined.

The problem consists of defining the number of goods moved from each facility $j \in \mathcal{J}$ to each customer $i \in \mathcal{I}$ to minimize the sum of the cost of opening facilities and the cost to ship the goods from the facilities to the customers. The constraints are the following:

- The total quantity of goods taken from an open facility cannot exceed its capacity;
- The total quantity of goods supplied to a customer must be exactly equal to its demand;
- The same facility cannot supply two incompatible customers.

Data Mining Application

In the proposed method, the mined patterns are frequent sets of pairs $\langle i, j \rangle$, i.e., sets of customer-facility assignments frequent in solutions of the elite set E . Since each decision variable x_{ij} in this problem represents an integer quantity of goods supplied to a customer i by a facility j , the search space reduction is achieved by constraining the decision variables' domains as a consequence of fixing a supplied quantity for each frequent assignment. Hence, after the frequent itemsets are mined, their assignments are filled with the corresponding minimum quantities in E , i.e., for each pair $\langle i, j \rangle$ in a frequent itemset, a supplied quantity q is set, which corresponds to the minimum positive supplied quantity for that pair among all solutions in E . Therefore, each final pattern is a set of assignments $\langle i, j, q \rangle$.

Patterns are used to reduce the size of the problem instance based on the assumption that their elements shall be part of the solution. In the particular case of reducing this problem, an extension to the problem formulation is needed, with the introduction of incompatibilities between customers and facilities, represented by a set Γ' , such that for each $\langle i, j \rangle \in \Gamma'$, i cannot be supplied by j . Hence, an additional set of constraints is introduced:

- An incompatible facility cannot supply a customer.

The reduction process works as follows. Given a problem instance $\mathcal{P}$ and a pattern p , the reduced instance $\mathcal{P}'$ is initialized as a copy of $\mathcal{P}$. Then, for each

assignment $\langle i, j, q \rangle$ in p , the following changes are applied to $\mathcal{P}'$: (i) decrease both s_j and d_i by q (a supply of q goods to customer i by facility j is fixed); (ii) set $f_j = 0$, because since facility j is already open, there is no additional cost to use it; and (iii) for each customer i' incompatible with i (from Γ), add a pair $\langle i', j \rangle$ to Γ' (facility j cannot supply any customer that is incompatible with i).

Consequently, the domain of each x_{ij} variable will be reduced since the customer's demands and facilities capacities are reduced, leading to diminished maximum values for x_{ij} . Also, additional constraints will fix the values of many other decision variables, effectively reducing the problem instance size.

Computational Results

Four methods have been compared in [11]: the MineReduce-based Multi-Start ILS (MR-MS-ILS); a Greedy Randomized Adaptive Search Procedure (GRASP); a Permutation Coded Evolutionary Algorithm (PcEA); and a Multistart Greedy (MG) heuristic.

The methods have been tested on artificial instances created for the competition. Instances have been partitioned into two sets, a *training* set with 20 instances, which was publicly available since the competition started so the competitors could use it for tuning their algorithms, and a *validation* set with 10 instances, which was kept hidden during the competition as it was used for defining the final ranking.

The comparisons are based on the best solution found by each method in a given time-out (t). Two different time-outs are used based on the number of facilities J in the problem instance. The first is the one proposed for the competition, which grants $10\sqrt{J}$ seconds for solving each instance, whereas the second grants J seconds per instance.

Tables 8 and 9 summarize the results for the first and the second time-outs, respectively, presenting the numbers of wins (the number of instances for which a method achieved the best results) and the average percentage difference (APD) to PcEA for the training and validation sets separately and for all of them together (global).

Additionally, Table 10 shows the average ranks throughout all runs (10×4) upon all instances for each combination of dataset and time-out, which was the criteria adopted in the competition for ranking the methods. Note that if a method had been better than all others in all runs, it would have obtained ranks 1, 2, ..., 10 for its ten runs, resulting in an average rank of 5.5 (the average of numbers 1, 2, ..., 10).

The results show that MR-MS-ILS was indisputably superior to the other methods. It achieved better solution quality for the vast majority of the instances and far better average ranks.

During the development of the proposed method, experiments comparing MR-MS-ILS to a version of it with the MineReduce-based components disabled (MS-ILS) were conducted to measure the effect of the MineReduce approach – using the training set instances with $t = 10\sqrt{J}$. Table 11 summarizes the results obtained in this comparison, showing that the MineReduce approach significantly improves the baseline method's performance.

Table 8 Summarized results for the MS-CFLP-CI ($t = 10\sqrt{J}$)

		PcEA			GRASP			MG			MR-MS-ILS		
		Best Cost	Avg. Cost		Best Cost	Avg. Cost		Best Cost	Avg. Cost		Best Cost	Avg. Cost	
Train. Set	Wins	–	–		3		3	2		2	15		15
	APD				– 76.79%	– 78.09%		– 76.78%	– 78.24%		– 80.21%	– 81.58%	
Valid. Set	Wins	–	–		1		1	3		3	6		6
	APD				– 76.77%	– 77.64%		– 77.14%	– 78.08%		– 80.34%	– 81.24%	
Global	Wins	–	–		4		4	5		5	21		21
	APD				– 76.79%	– 77.94%		– 76.90%	– 78.19%		– 80.25%	– 81.46%	

Table 9 Summarized results for the MS-CFLP-CI ($t = J$)

		PcEA		GRASP		MG		MR-MS-ILS	
		Best Cost	Avg. Cost	Best Cost	Avg. Cost	Best Cost	Avg. Cost	Best Cost	Avg. Cost
Train. Set	Wins	–	–	3	3	1	2	16	15
	APD			– 76.71%	– 77.82%	– 76.56%	– 77.82%	– 80.18%	– 81.35%
Valid. Set	Wins	–	–	1	1	2	2	7	7
	APD			– 76.63%	– 77.69%	– 76.73%	– 77.92%	– 80.36%	– 81.37%
Global	Wins	–	–	4	4	3	4	23	22
	APD			– 76.68%	– 77.78%	– 76.62%	– 77.86%	– 80.24%	– 81.35%

Table 10 Average ranks obtained by the methods

	Train. Set		Valid. Set	
	$t = 10\sqrt{J}$	$t = J$	$t = 10\sqrt{J}$	$t = J$
MR-MS-ILS	8.3	7.7	9.2	8.7
MG	17.7	18.4	16.4	16.9
GRASP	20.5	20.4	20.9	20.8
PcEA	35.5	35.5	35.5	35.5

Table 11 Summarized results for the MS-CFLP-CI (MR-MS-ILS vs. MS-ILS)

	MS-ILS		MR-MS-ILS	
	Best Cost	Avg. Cost	Best Cost	Avg. Cost
Wins	–	–	19	20
APD			– 2.42%	– 2.46%

Conclusion

This chapter presents some ideas for incorporating data mining in local search heuristics. Some applications have been described, showing that heuristics can benefit from data mining by obtaining better solutions in shorter computational times.

These results motivate the use of data mining in other metaheuristics, where the patterns can also be used to guide the search. Data mining can also be used in exact algorithms like branch-and-bound, where the values of some variables could be set based on the patterns extracted from previously found solutions.

Analyzing the obtained results, it seems reasonable to expect that incorporating data mining techniques into SLS heuristics can be helpful in general. Wolpert and Macread [18] presented a series of No Free Lunch theorems which establish that for any algorithm, the high performance on a class of problems is offset by the low performance of another class. So, they conclude that all black-box optimization algorithms perform exactly equally well. However, some subsequent studies [3, 6] have shown that these theorems apply to some specific classes of problems and that is expected that real-world problems are not of this particular class. Therefore, it is reasonable to expect that the good results obtained using the data mining techniques in the presented problems may also be achieved in other real-world problems.

Cross-References

- [Evolutionary Algorithms](#)
- [Genetic Algorithms](#)
- [Heuristics in Vehicle Routing](#)
- [Innovative Applications of Metaheuristics to Supervised Machine Learning](#)
- [Iterated Local Search](#)
- [Multi-start Methods](#)
- [Restart Strategies](#)

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

- Berthold T (2013) Measuring the impact of primal heuristics. *Oper Res Lett* 41(6):611–614. <https://doi.org/10.1016/j.orl.2013.08.007>
- Duhamel C, Lacomme P, Prodhon C (2010) A GRASPxELS with depth first search split procedure for the HVRP. Technical report, Laboratoire d'Informatique, de Modélisation et d'Optimisation des Systèmes
- García-Martínez C, Rodríguez FJ, Lozano M (2012) Arbitrary function optimisation with metaheuristics. *Soft Comput* 16(12):2115–2133. <https://doi.org/10.1007/s00500-012-0881-x>
- Han J, Kamber M, Pei J (2012) Data mining: concepts and techniques, 3rd edn. Morgan Kaufmann. <https://doi.org/10.1016/C2009-0-61819-5>
- Hoos HH, Stützle T (2005) Stochastic local search: foundations and applications. Elsevier, San Francisco. <https://doi.org/10.1016/B978-1-55860-872-6.X5016-1>
- Igel C, Toussaint M (2003) On classes of functions for which no free lunch results hold. *Inf Process Lett* 86(6):317–321. [https://doi.org/10.1016/S0020-0190\(03\)00222-9](https://doi.org/10.1016/S0020-0190(03)00222-9)
- de Lima Martins S, Rosseti I, Plastino A (2018) Data mining in stochastic local search. In: Martí R, Pardalos PM, Resende MGC (eds) Handbook of heuristics. Springer International Publishing, Cham, pp 39–87. https://doi.org/10.1007/978-3-319-07124-4_11
- Maia MRH, Plastino A, Penna PHV (2018) Hybrid data mining heuristics for the heterogeneous fleet vehicle routing problem. *RAIRO-Oper Res* 52(3):661–690. <https://doi.org/10.1051/ro/2017072>
- Maia MRH, Plastino A, Penna PHV (2020) MineReduce: an approach based on data mining for problem size reduction. *Comput Oper Res* 122:104995. <https://doi.org/10.1016/j.cor.2020.104995>
- Maia MRH, Plastino A, Souza US (2024) An improved hybrid genetic search with data mining for the CVRP. *Networks* 83(4):692–711. <https://doi.org/10.1002/net.22213>
- Maia MRH, Reula M, Parreño-Torres C, Vuppuluri PP, Plastino A, Souza US, Ceschia S, Pavone M, Schaerf A (2023) Metaheuristic techniques for the capacitated facility location problem with customer incompatibilities. *Soft Comput* 27(8):4685–4698. <https://doi.org/10.1007/s00500-022-07600-z>
- Penna PHV, Subramanian A, Ochi LS (2013) An iterated local search heuristic for the heterogeneous fleet vehicle routing problem. *J Heurist* 19(2):201–232. <https://doi.org/10.1007/s10732-011-9186-y>
- Sörensen K, Arnold F, Pálhazy Cuervo D (2019) A critical analysis of the “improved Clarke and Wright savings algorithm”. *Int Trans Oper Res* 26(1):54–63. <https://doi.org/10.1111/itor.12443>
- Talbi EG (2021) Machine learning into metaheuristics: a survey and taxonomy. *ACM Comput Surv* 54(6). <https://doi.org/10.1145/3459664>
- Tan PN, Karpatne A, Steinbach M, Kumar V (2019) Introduction to data mining, 2nd edn. Pearson, Deutschland
- Vidal T (2022) Hybrid genetic search for the CVRP: open-source implementation and SWAP* neighborhood. *Comput Oper Res* 140:105643. <https://doi.org/10.1016/j.cor.2021.105643>
- Witten IH, Frank E (2011) Data mining: practical machine learning tools and techniques, 3rd edn. Morgan Kaufmann. <https://doi.org/10.1016/C2009-0-19715-5>
- Wolpert D, Macready W (1997) No free lunch theorems for optimization. *IEEE Trans Evol Comput* 1(1):67–82. <https://doi.org/10.1109/4235.585893>
- Zaki MJ, Meira Jr W (2014) Data mining and analysis: fundamental concepts and algorithms. Cambridge University Press. <https://doi.org/10.1017/CBO9780511810114>